

Healthcare Management System – Salesforce Project

1. Project Overview

The Healthcare Management System is a Salesforce-based application designed to manage patients, doctors, appointments, medical requests, and medical records in a centralized platform.

The main objective of this project is to simplify healthcare-related operations by maintaining patient information, managing doctor details, scheduling appointments, automating follow-up activities, preventing duplicate appointments, and providing quick access to patient and appointment information. The project demonstrates the use of Salesforce Custom Objects, Apex, Triggers, Trigger Handlers, Flows, Lightning Web Components (LWC), REST API Integration, Reports, and Dashboards.

2. Project Approach

The project was developed by following a modular approach where different Salesforce features were used for different business requirements.

Major areas implemented:

- Custom Objects for healthcare data management
- Relationships between Patients, Doctors, and Appointments
- Salesforce Flows for business process automation
- Apex Triggers for complex business logic
- Trigger Handler for maintaining clean and reusable trigger logic
- Lightning Web Components for custom user interfaces
- REST API for integration
- Reports and Dashboards for data analysis and visualization

The implementation focuses on reducing manual work, improving data accuracy, and providing a better user experience for healthcare-related operations.

3)Screenshot Sections

Patient

The Patient object stores important patient information such as:

- Patient Name
- Mobile Number
- Email
- Other relevant patient details

This object acts as the primary source for maintaining patient information

The screenshot displays the Salesforce Lightning interface for a Healthcare Management System. The top navigation bar includes a search bar, a "Developer Edition" badge, and a menu with options like Home, Patients, Doctors, Appointments, Medical Requests, Medical Records, Dashboards, and More. The main content area shows the "Patients" list, which is filtered to "All" and contains 7 items. The list is sorted by Patient ID and updated a few seconds ago. The list table has columns for Patient ID, Patient Name, Blood Group, and Gender. Each row represents a patient with a unique ID, name, blood group, and gender.

	☐ Patient ID ↑	Patient Name	Blood Group	Gender	
1	☐ PI-01	Kaviya	A+	Female	▼
2	☐ PI-02	Rahul	AB-	Male	▼
3	☐ PI-03	Meena	A-	Female	▼
4	☐ PI-04	Priya	B+	Female	▼
5	☐ PI-05	Kishore	O+	Male	▼
6	☐ PI-06	Balu	AB-	Male	▼
7	☐ PI-07	Sakthi	O-	Male	▼

Doctor

The Doctor object stores doctor-related information such as:

- Doctor Name
- Specialization
- Contact information
- Doctor availability/details

The screenshot displays the Salesforce Lightning interface for the 'Doctors' object. The top navigation bar shows the 'Developer Edition' label and a search bar. The sidebar on the left contains the 'Healthcare Management S...' label. The main content area features the 'Doctors' header with a filter dropdown set to 'All'. Below the header, there are buttons for 'New', 'Import', 'Change Owner', 'Printable View', and 'Assign Label'. A search bar labeled 'Search this list...' is also present. The table below lists 3 items, sorted by Doctor ID. The table has columns for 'Doctor ID', 'Doctor Name', and 'Email'.

	☐ Doctor ID ↑	Doctor Name	Email
1	☐ DI-01	Kumar	kumar45@gmail.com
2	☐ DI-02	Santhosh	santhosh67@gmail.com
3	☐ DI-03	Arun	arun78@gmail.com

Appointment

The Appointment object is used to manage patient appointments with doctors.

It contains information such as:

- Patient
- Doctor
- Appointment Date
- Appointment Time
- Status
- Reason for Appointment

orgfarm-bc43689a0a-dev-ed.develop.lightning.force.com/lightning/o/Appointment__c/list?filterName=All

Developer Edition

Search...

Healthcare Management S... Home Patients Doctors Appointments Medical Requests Medical Records Dashboards More

Appointments All

New Import Change Owner Printable View Assign Label

Search this list...

14 items • Sorted by Appointment Number • Updated a few seconds ago

	☐ Appointment Num... ↑	Appointment Date	Appointment Time	Doctor Name	Status	
1	☐ AI-01	8/21/2026	10:00 AM	DI-03	Scheduled	▼
2	☐ AI-02	8/14/2026	12:30 AM	DI-02	Scheduled	▼
3	☐ AI-03	8/28/2026	12:45 AM	DI-01	Scheduled	▼
4	☐ AI-04	8/21/2026	8:00 AM	DI-03	Scheduled	▼
5	☐ AI-05	8/21/2026	12:15 AM	DI-01	Scheduled	▼
6	☐ AI-06	8/21/2026	12:30 AM	DI-03	Scheduled	▼
7	☐ AI-07	8/21/2026	12:30 PM	DI-02	Scheduled	▼
8	☐ AI-08	8/14/2026	12:45 AM	DI-02	Scheduled	▼

Medical Request

The Medical Request object is used to manage medical-related requests and follow-up activities associated with patients.

The screenshot shows the Salesforce interface for the Medical Requests object. The browser address bar displays the URL: `orgfarm-bc43689a0a-dev-ed.develop.lightning.force.com/lightning/o/Medical_Request__c/list?filterName=All`. The page header includes the Salesforce logo, a search bar, and navigation tabs: Home, Patients, Doctors, Appointments, Medical Requests (selected), Medical Records, Dashboards, and More. The main content area is titled "Medical Requests" and shows a list of 4 items. The list is sorted by Medical Request Number and updated a few seconds ago. The list includes columns for Medical Request Number, Appointment, and Status. The data rows show four completed requests with IDs MI-01, MI-02, MI-03, and MI-04, all associated with appointment AI-01.

	Medical Request Number ↑	Appointment	Status
1	MI-01	AI-06	Completed
2	MI-02	AI-01	Completed
3	MI-03	AI-01	Completed
4	MI-04	AI-01	Completed

Medical Records

The Medical Records object is used to maintain patient medical information and history in Salesforce.

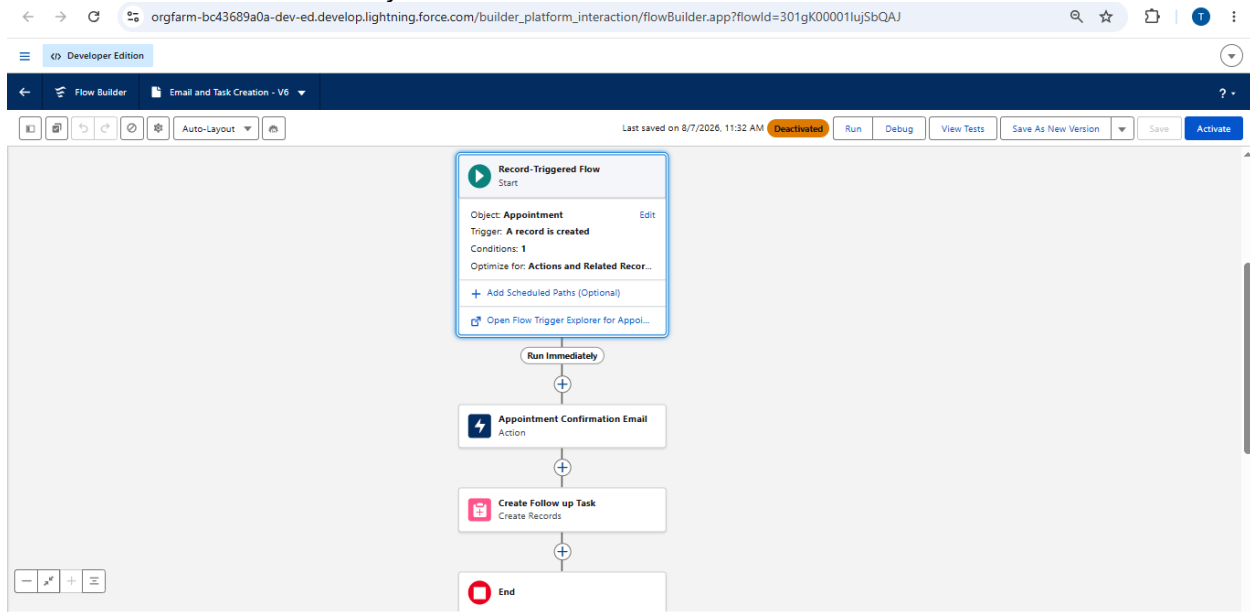
The screenshot shows the Salesforce interface for the Medical Records object. The browser address bar displays the URL: `orgfarm-bc43689a0a-dev-ed.develop.lightning.force.com/lightning/o/Medical_Record__c/list?filterName=All`. The page header includes the Salesforce logo, a search bar, and navigation tabs: Home, Patients, Doctors, Appointments, Medical Requests, Medical Records (selected), Dashboards, and More. The main content area is titled "Medical Records" and shows a list of 1 item. The list is sorted by Medical Record Number and updated a few seconds ago. The list includes columns for Medical Record Number, Appointment, Doctor Name, and Visit Date. The data row shows a single record with ID MRI-01, associated with appointment AI-01, doctor DI-03, and visit date 8/21/2026.

	Medical Record Number ↑	Appointment	Doctor Name	Visit Date
1	MRI-01	AI-01	DI-03	8/21/2026

Flow 1 – Task and Email Automation

When an Appointment is created:

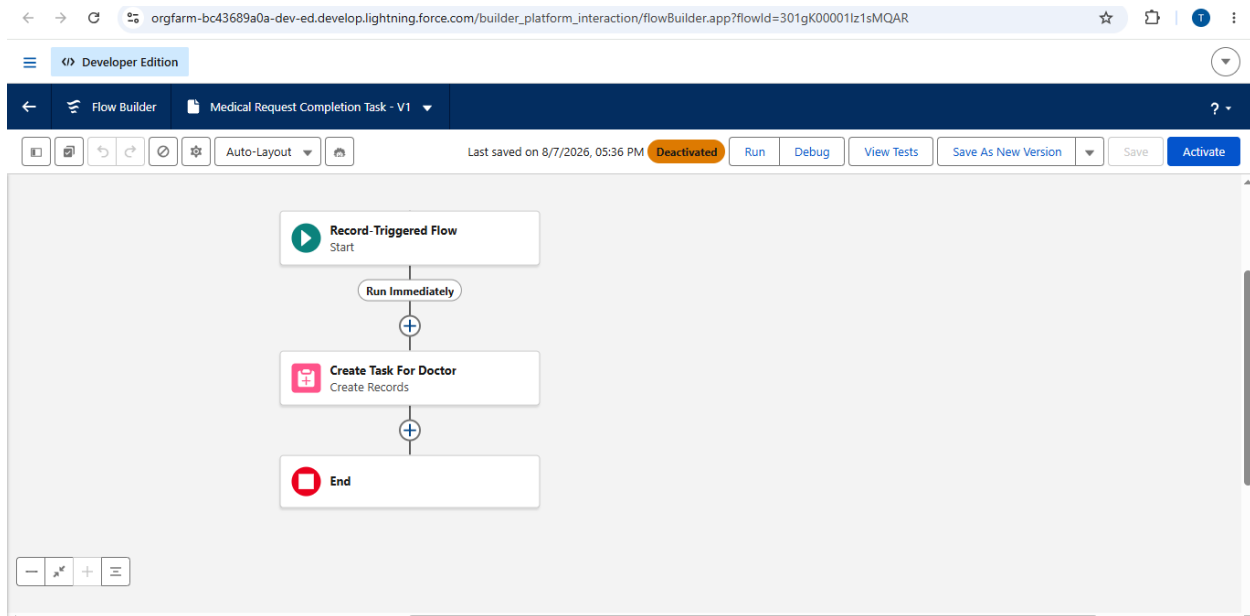
- A Task is automatically created for the Doctor.
- An Email is automatically sent to the Patient.



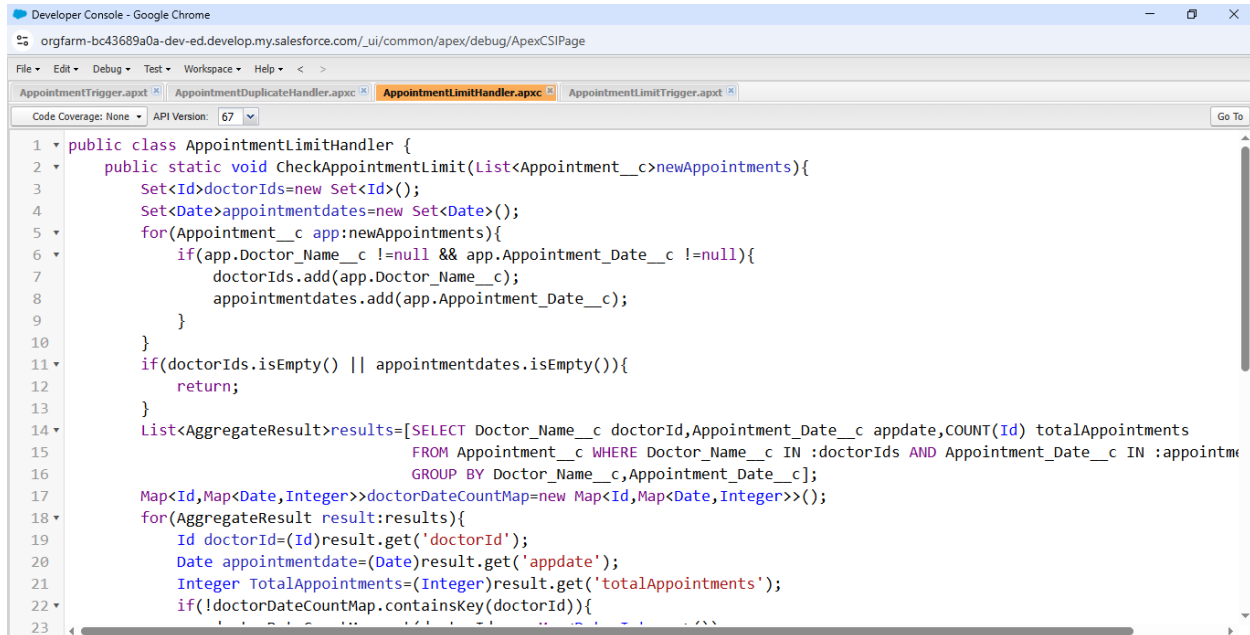
Flow 2 – Medical Request Automation

When a Medical Request status changes to Completed:

- A Task is automatically created for the Doctor.

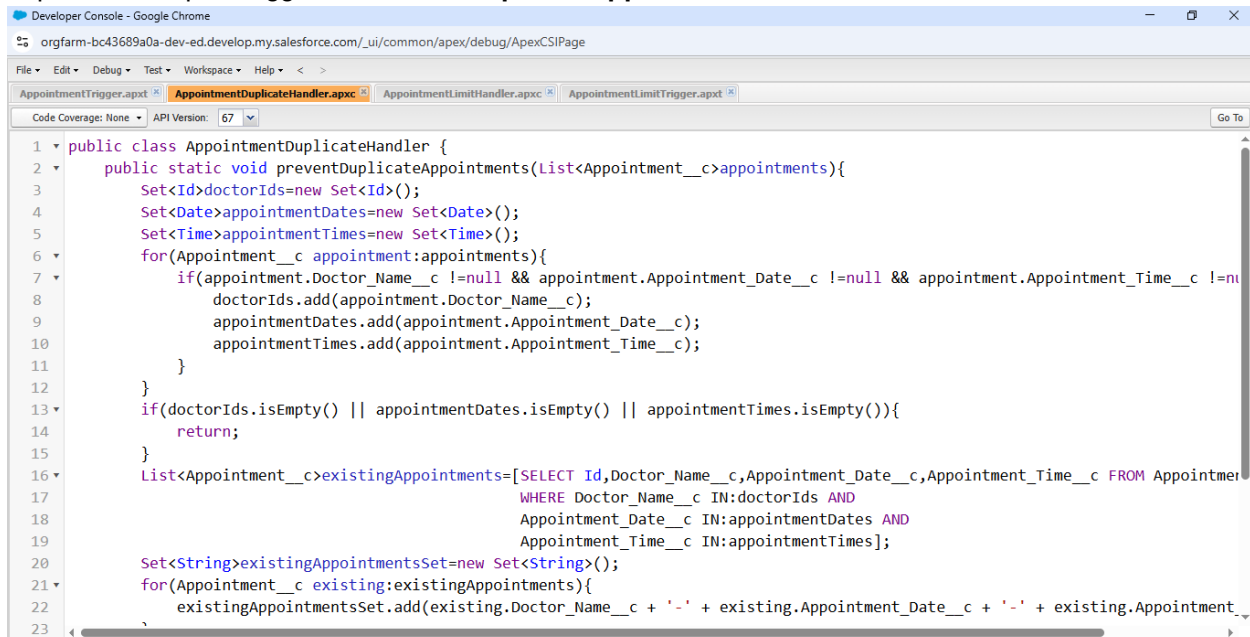


Implemented **Apex Trigger** with Trigger Handler to manage appointment business rules.
Limits a doctor appointment to a maximum of 20 appointments per day.
Displays an error message when the limit is exceeded.



```
1 public class AppointmentLimitHandler {
2     public static void CheckAppointmentLimit(List<Appointment__c> newAppointments){
3         Set<Id> doctorIds = new Set<Id>();
4         Set<Date> appointmentDates = new Set<Date>();
5         for(Appointment__c app : newAppointments){
6             if(app.Doctor_Name__c != null && app.Appointment_Date__c != null){
7                 doctorIds.add(app.Doctor_Name__c);
8                 appointmentDates.add(app.Appointment_Date__c);
9             }
10        }
11        if(doctorIds.isEmpty() || appointmentDates.isEmpty()){
12            return;
13        }
14        List<AggregateResult> results = [SELECT Doctor_Name__c doctorId, Appointment_Date__c appdate, COUNT(Id) totalAppointments
15                                       FROM Appointment__c WHERE Doctor_Name__c IN :doctorIds AND Appointment_Date__c IN :appointmentDates
16                                       GROUP BY Doctor_Name__c, Appointment_Date__c];
17        Map<Id, Map<Date, Integer>> doctorDateCountMap = new Map<Id, Map<Date, Integer>>();
18        for(AggregateResult result : results){
19            Id doctorId = (Id) result.get('doctorId');
20            Date appointmentDate = (Date) result.get('appdate');
21            Integer totalAppointments = (Integer) result.get('totalAppointments');
22            if(!doctorDateCountMap.containsKey(doctorId)){
23                Map<Date, Integer> doctorDateCount = new Map<Date, Integer>();
```

Implemented Apex Trigger to Prevents **duplicate appointments** for the same Doctor, Date, and Time.



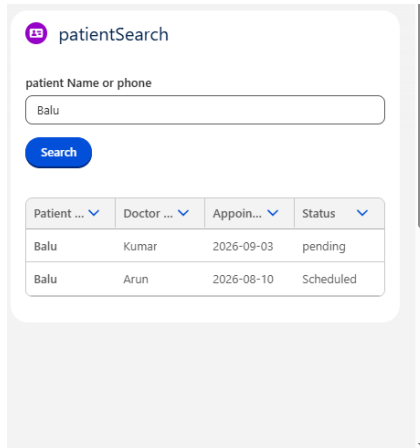
```
1 public class AppointmentDuplicateHandler {
2     public static void preventDuplicateAppointments(List<Appointment__c> appointments){
3         Set<Id> doctorIds = new Set<Id>();
4         Set<Date> appointmentDates = new Set<Date>();
5         Set<Time> appointmentTimes = new Set<Time>();
6         for(Appointment__c appointment : appointments){
7             if(appointment.Doctor_Name__c != null && appointment.Appointment_Date__c != null && appointment.Appointment_Time__c != null){
8                 doctorIds.add(appointment.Doctor_Name__c);
9                 appointmentDates.add(appointment.Appointment_Date__c);
10                appointmentTimes.add(appointment.Appointment_Time__c);
11            }
12        }
13        if(doctorIds.isEmpty() || appointmentDates.isEmpty() || appointmentTimes.isEmpty()){
14            return;
15        }
16        List<Appointment__c> existingAppointments = [SELECT Id, Doctor_Name__c, Appointment_Date__c, Appointment_Time__c FROM Appointment__c
17                                                    WHERE Doctor_Name__c IN :doctorIds AND
18                                                    Appointment_Date__c IN :appointmentDates AND
19                                                    Appointment_Time__c IN :appointmentTimes];
20        Set<String> existingAppointmentsSet = new Set<String>();
21        for(Appointment__c existing : existingAppointments){
22            existingAppointmentsSet.add(existing.Doctor_Name__c + '-' + existing.Appointment_Date__c + '-' + existing.Appointment_Time__c);
23        }
24        for(Appointment__c appointment : appointments){
25            String key = appointment.Doctor_Name__c + '-' + appointment.Appointment_Date__c + '-' + appointment.Appointment_Time__c;
26            if(existingAppointmentsSet.contains(key)){
27                // Duplicate appointment found
28            }
29        }
30    }
31}
```

LWC 1 – Patient Search

A Patient Search component was developed to allow users to search for patients using:

- Patient Name
- Mobile Number

After performing the search, the matching patient and appointment-related information is displayed in a structured table.



patientSearch

patient Name or phone

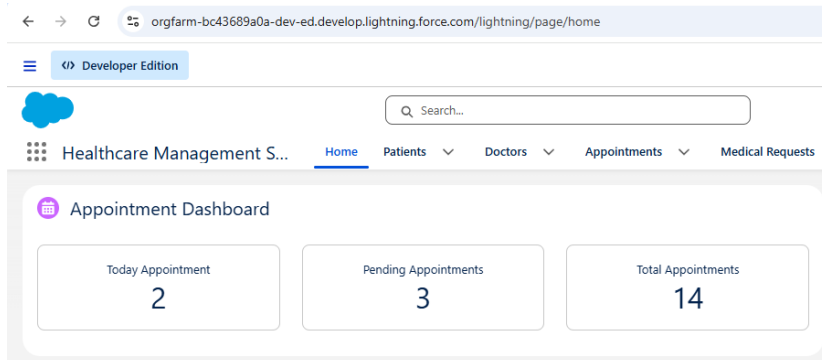
Balu

Search

Patient ...	Doctor ...	Appoin...	Status
Balu	Kumar	2026-09-03	pending
Balu	Arun	2026-08-10	Scheduled

LWC 2 – Appointment Dashboard

An Appointment Dashboard LWC was also developed to provide a custom interface for viewing appointment-related information like today appointments, total appointments and pending appointments.



orgfarm-bc43689a0a-dev-ed.develop.lightning.force.com/lightning/page/home

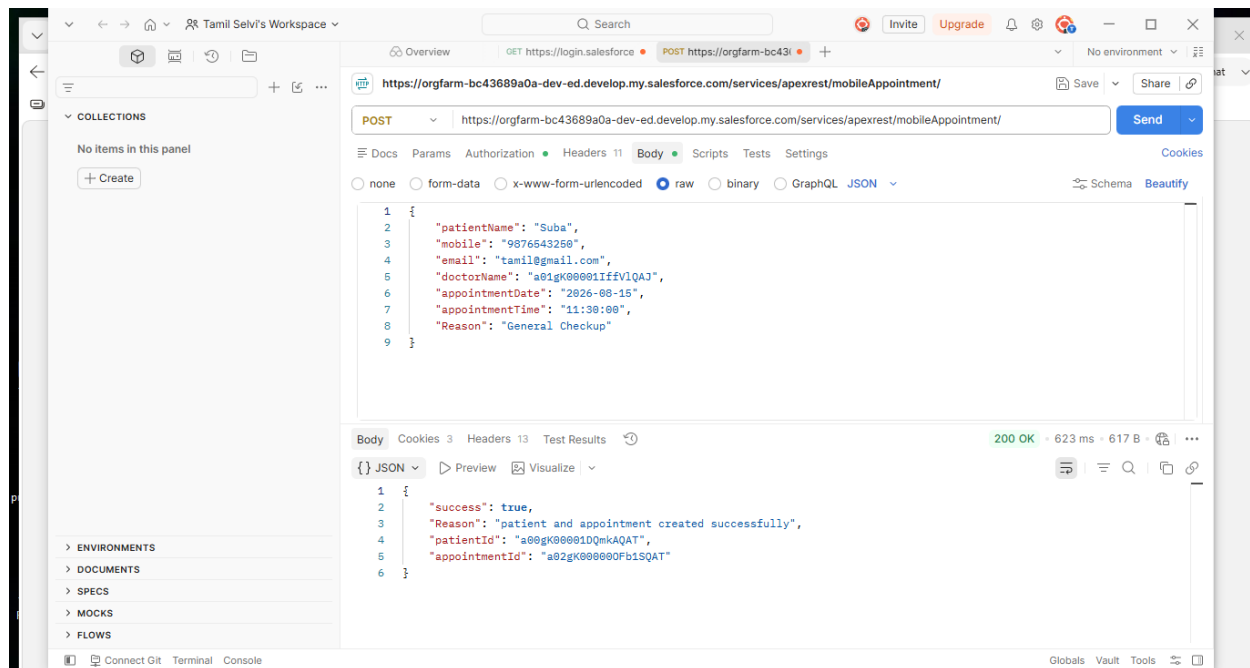
Developer Edition

Search...

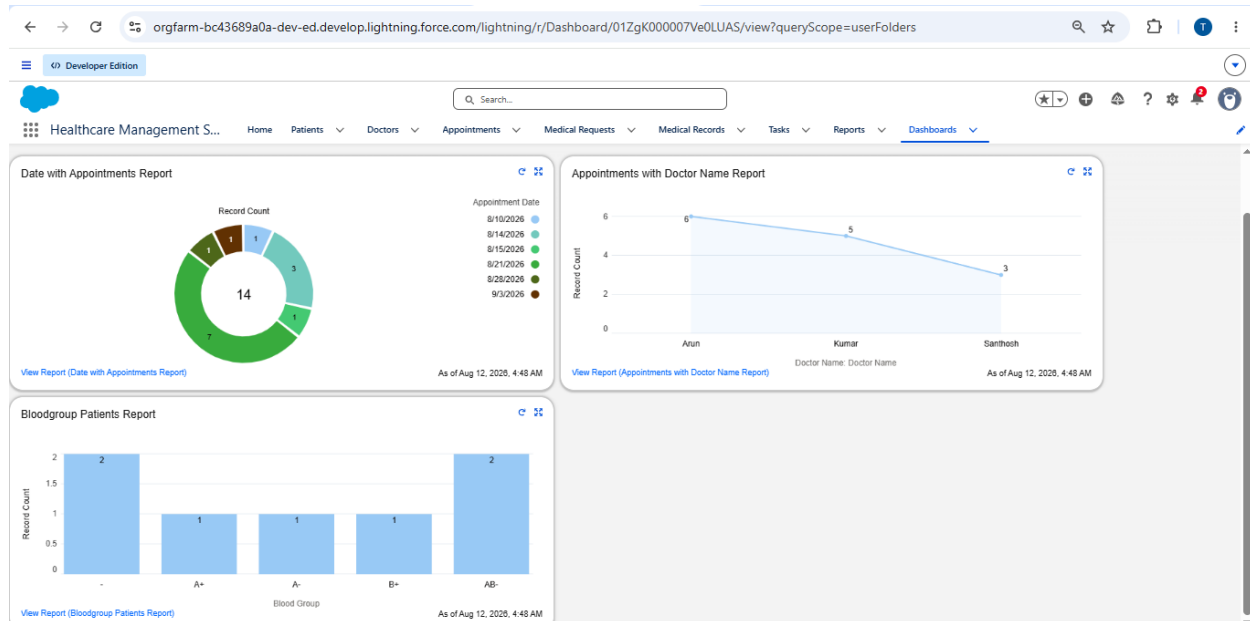
Healthcare Management S... Home Patients Doctors Appointments Medical Requests

Appointment Dashboard

Today Appointment	Pending Appointments	Total Appointments
2	3	14



A backend application is integrated with Salesforce using **REST API**. When a patient or appointment is created in the backend application, the corresponding **Patient and Appointment records are automatically created in Salesforce**.



Created a **Healthcare Dashboard** to monitor important appointment information.

- **Blood Group** – Displays patient distribution by blood group.
- **Date** – Shows the number of appointments based on date.
- **Doctor Name** – Shows the number of appointments handled by each doctor.

This dashboard provides a quick overview of patient and appointment data.

Conclusion

The Healthcare Management System demonstrates how Salesforce can be used to build a complete business application by combining declarative and programmatic development.

This project provided practical experience in designing Salesforce data models, implementing automation, developing custom business logic, building user interfaces, integrating external systems, and presenting business data through reports and dashboards.